A COMPLETE PRACTITIONER'S GUIDE

# Advanced Retrieval-Augmented Generation

From First Principles to Production-Grade
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE SEVEN

## Advanced RAG Patterns

Beginner to Industry Level • Assumes only basic Python

MODULE SEVEN

# Advanced RAG Patterns

*Modules 5 and 6 sharpened the retriever. This module redesigns the entire RAG pipeline. You will study the named architectures that the field has converged on — RAG Fusion and Reciprocal Rank Fusion, HyDE, Corrective RAG, Self-RAG, Graph RAG, and Multi-modal RAG. Each is a complete pattern that solves a specific failure of naive RAG: query-answer mismatch, blind trust in bad retrieval, unverified answers, multi-hop reasoning, and non-text content. By the end you will know not just how each pattern works, but precisely when to reach for it.*

> **WHERE WE ARE**
>
> So far you have built RAG one component at a time: load, split, embed, store, retrieve. This module zooms back out to the *whole pipeline* and asks how to re-architect it for reliability. Several patterns here — especially Corrective RAG and Self-RAG — involve loops and decisions that are awkward in a linear chain. Their *concepts* are taught here; their natural *implementation* uses LangGraph, the subject of Module 8.

## 7.1 Introduction

Everything you have built so far is what the field calls **naive RAG** — and "naive" is not an insult, it is a precise technical label. Naive RAG is the straight-line pipeline of Module 1: take the query, retrieve once, stuff the results into a prompt, generate an answer. It is the correct thing to build first, and for many applications it is genuinely enough.

But naive RAG has a fixed, predictable set of failure modes, and once you have deployed a RAG system to real users you will meet every one of them:

- **Query-answer mismatch.** A short question and a long answer passage are different *kinds* of text. The question's embedding may simply not sit close to the answer's, so retrieval misses it.
- **Blind trust in retrieval.** Naive RAG uses whatever the retriever returns, even when the retriever returned garbage. There is no check, no second opinion, no fallback.
- **Unverified generation.** The model produces an answer, and naive RAG ships it — with no step that asks "is this answer actually supported by the retrieved evidence?"

- **No multi-hop reasoning.** Questions that require connecting facts across several documents — "*which projects are led by managers who joined before 2020?*" — defeat naive RAG, because it retrieves chunks independently and never connects them.
- **Text-only.** Naive RAG cannot see the diagram, the chart, or the scanned table that holds the answer.

**Advanced RAG patterns** are the field's answers to these failures. Each is a named, reusable architecture — a specific way of re-wiring the pipeline so that one of these weaknesses is removed. RAG Fusion and HyDE attack query-answer mismatch. Corrective RAG and Self-RAG add verification and self-correction. Graph RAG enables multi-hop reasoning. Multi-modal RAG sees images and tables.

This module is your catalogue of these patterns. For each, you will learn the failure it fixes, how it works internally, and — most importantly — *when* it is the right choice. Knowing the patterns is good; knowing which to deploy for a given problem is what makes you a RAG architect.

## 7.2 Why This Topic Matters

> **WHY THIS MATTERS**
>
> The gap between a RAG *prototype* and a RAG *product* is largely the gap this module closes. A prototype that works in a demo is naive RAG. A product that holds up against real, messy, adversarial usage almost always incorporates one or more of these patterns. They are also the heart of senior RAG interviews: "what is HyDE?", "how does Corrective RAG differ from Self-RAG?", and "when would you use Graph RAG?" are exactly the questions that distinguish a candidate who has read about RAG from one who has architected it.

Four concrete reasons this module repays close study:

1. **It names the failure modes.** Once you can name *why* naive RAG failed on a given query, you can choose the exact pattern that fixes it instead of guessing.
2. **These patterns are the production standard.** Real enterprise RAG systems are built from RAG Fusion, re-ranking, corrective loops, and — increasingly — graph and multi-modal retrieval.
3. **It introduces self-correcting AI.** Corrective RAG and Self-RAG are your first encounter with systems that *evaluate and improve their own behaviour* — the conceptual seed of the agents in Module 8.
4. **It is a vocabulary you must have.** "RAG Fusion," "HyDE," "CRAG," "Graph RAG" are standard terms in the field. You need to speak this language fluently.

## 7.3 A Beginner-Friendly Explanation

Think of naive RAG as a junior researcher with one rigid habit: *look it up once, trust whatever comes back, write the answer.* Advanced RAG patterns are the better habits that turn that junior researcher into a senior one. Here they are, in plain language.

**Searching several ways and combining the results (RAG Fusion).** A senior researcher does not run one search. They run several — phrasing the question different ways — and then *intelligently combine* the result lists, trusting most the sources that showed up across multiple searches. The "intelligently combine" step has a name: **Reciprocal Rank Fusion**.

**Imagining the answer before searching (HyDE).** Here is a subtle, clever habit. Before searching, the researcher first *writes a rough draft of what they think the answer might look like* — even though parts of the draft may be wrong. Then they search using that draft. Why? Because a draft answer *looks like* a real answer passage far more than a short question does. Searching with a fake answer finds real answers better. This is **HyDE**.

**Double-checking the sources, and going elsewhere if they are bad (Corrective RAG).** A senior researcher does not blindly trust the first sources they find. They *evaluate* them: "are these actually relevant and reliable?" If the sources are good, they proceed. If the sources are bad, they do not just use them anyway — they go *somewhere else*, perhaps to the open web. This self-checking, fallback-using habit is **Corrective RAG**.

**Critiquing your own draft against the evidence (Self-RAG).** Before submitting, the senior researcher re-reads their own draft and asks: "*Is every claim here actually backed by my sources? Is this answer even useful?*" If not, they revise. They also have the judgment to know when a question is so simple that no research is needed at all. This self-reflective, adaptive habit is **Self-RAG**.

**Using a map of how facts connect (Graph RAG).** Some questions are not "find a fact" but "connect facts" — "*who works under managers hired before 2020?*" A senior researcher answers these using a *map of relationships* between people, projects, and dates, not just a pile of documents. Building and searching such a map is **Graph RAG**.

**Reading the pictures, not just the words (Multi-modal RAG).** Finally, a senior researcher reads the charts, diagrams, and tables — not only the prose. A RAG system that can do the same is **Multi-modal RAG**.

That is the whole module: six professional research habits, each fixing one weakness of the naive approach.

**DEFINITION**

An **advanced RAG pattern** is a named, reusable re-architecture of the RAG pipeline that removes a specific failure mode of naive RAG. Patterns fall into two broad families: **retrieval-enhancement** patterns (RAG Fusion, HyDE, Graph RAG, Multi-modal RAG) that improve *what* is retrieved, and **self-correcting / adaptive** patterns (Corrective RAG, Self-RAG) that add steps where the system *evaluates and fixes its own behaviour*.

## 7.4 Real-World Analogy: The Junior and the Senior Researcher

**REAL-WORLD ANALOGY**

Two researchers are asked the same hard question.

The **junior** (naive RAG) types the question into a search box, takes the first few results, and writes up whatever they say. Fast — and fragile. If the search missed the key source, the junior never knows. If the sources contradict each other, the junior does not notice. If the answer needs a chart, the junior ignores the chart.

The **senior** researcher works completely differently. They search several ways and cross-reference. They sketch the expected answer first to search more effectively. They scrutinise their sources and, if those sources are weak, they go find better ones. They critique their own draft against the evidence before submitting. They consult a relationship map for questions about how things connect. And they read every figure and table, not just the text.

Same question, same library — a dramatically more reliable answer, because the senior researcher's *process* is richer. Advanced RAG patterns are exactly that richer process, encoded into the pipeline.

## 7.5 Core Concepts: From Naive to Advanced RAG

It is worth fixing the contrast precisely. **Naive RAG** is a *linear, single-pass, blind-trust* pipeline:

```
query → retrieve once → augment → generate → answer
(no checking, no looping, no adaptation)
```

**Advanced RAG** breaks each of those three properties:

- It is no longer strictly *linear* — patterns add branches and loops.
- It is no longer *single-pass* — patterns retrieve multiple times, or retry.
- It is no longer *blind-trust* — patterns add evaluation steps that grade retrieval and grade generation.

The patterns split cleanly into the two families named in the definition above. **Retrieval-enhancement patterns** make the *retrieval* better — they change what comes back. **Self-correcting and adaptive patterns** add *control flow* — branches, loops, and self-evaluation that make the pipeline respond to its own intermediate results. Hold this two-family split in mind; it is how you will organise everything that follows, and it is also why the self-correcting patterns point naturally toward Module 8: a pipeline that evaluates itself and decides what to do next is, in essence, an agent.

## 7.6 RAG Fusion and Reciprocal Rank Fusion

### 7.6.1 The Idea

In Module 6 you met **multi-query retrieval**: generate several phrasings of a query, search each, and *pool* the results. Pooling is crude — it just unions the lists and removes duplicates, treating a chunk that appeared first in three lists exactly like a chunk that appeared last in one.

**RAG Fusion** improves on this with a smarter combination step. It is multi-query retrieval *plus* an intelligent fusion algorithm called **Reciprocal Rank Fusion (RRF)**. RRF does not just pool the lists — it *re-ranks* the combined results so that chunks which ranked highly across *multiple* query variations rise to the top. It rewards **consensus**: a chunk that several different phrasings all agree is relevant is very probably relevant.

### 7.6.2 Reciprocal Rank Fusion, Precisely

RRF — first introduced in Module 5 as the engine behind hybrid search — is a method for merging several ranked lists into one. Its defining trick: **it ignores raw scores and uses only rank position.** This is what makes it robust, because rank is comparable across lists even when scores (a cosine similarity vs a BM25 score vs a different query's results) are not.

The RRF score of a document is the sum, across every list it appears in, of a *reciprocal rank* term:

```
RRF_score(document) =   Σ        1
                      all lists  ─────────
                                 k + rank

rank  = the document's 1-indexed position in that list
k     = a smoothing constant, conventionally 60
```

The intuition of the formula: a document at rank 1 contributes `1/(60+1) ≈ 0.0164`; at rank 10 it contributes `1/(60+10) ≈ 0.0143`. Appearing high is worth more than appearing low — but appearing in *several* lists, even at middling ranks, accumulates a large total. The constant `k` (≈ 60) softens the difference between the very top ranks so a single list cannot completely dominate. Documents are then sorted by their summed RRF score.

```python
def reciprocal_rank_fusion(ranked_lists, k=60):
    """Merge several ranked lists of Documents into one fused ranking."""
    scores, doc_map = {}, {}
    for ranked_list in ranked_lists:
        for rank, doc in enumerate(ranked_list, start=1):
            key = doc.page_content                # a stable identity
            doc_map[key] = doc
            scores[key] = scores.get(key, 0.0) + 1.0 / (k + rank)
    ranked = sorted(scores.items(), key=lambda kv: kv[1], reverse=True)
    return [doc_map[key] for key, _ in ranked]
```

So **RAG Fusion = generate query variations → retrieve a ranked list for each → fuse all lists with RRF → take the top results.** The pay-off is both higher recall (multiple queries cast a wider net) and better precision (RRF promotes consensus chunks to the top).

**INTERVIEW INSIGHT**

"What is the difference between multi-query retrieval and RAG Fusion?" — both generate several query variations; the difference is the *combination step*. Multi-query simply pools and de-duplicates the results. RAG Fusion fuses them with **Reciprocal Rank Fusion**, which re-ranks the merged set by rewarding chunks that ranked highly across *multiple* queries. Be ready to state the RRF formula — sum of `1/(k + rank)` over all lists — and the key reason it works: it uses *rank*, not raw score, so it merges lists whose scores are not comparable.

# 7.7 HyDE — Hypothetical Document Embeddings

## 7.7.1 The Problem: Query-Answer Asymmetry

Here is a subtle flaw in naive semantic retrieval. You embed the *question* and search for chunks whose embeddings are nearest. But a question and its answer are *different kinds of text*. A question is short, interrogative, and often vague ("*refund window?*"). An answer passage is longer, declarative, and detailed ("*Customers may return any item within 30 days of delivery for a full refund, provided…*"). Their embeddings can land surprisingly far apart simply because they differ in *form*, even though they are about the same topic. This is **query-answer asymmetry**, and it quietly hurts recall.

## 7.7.2 The HyDE Solution

**HyDE — Hypothetical Document Embeddings** — is a clever, almost paradoxical fix. Instead of searching with the question, HyDE:

1. Asks the LLM to *generate a hypothetical answer* to the question — a plausible passage of what the answer *might* say. The LLM is told not to hedge: just write a confident, answer-shaped paragraph.
2. **Embeds that hypothetical answer** instead of the question.
3. Uses the hypothetical answer's embedding to search the vector store.

The insight: a hypothetical answer, even an imperfect one, *looks like a real answer passage* — same form, same length, same declarative style, similar vocabulary. Its embedding therefore lands much closer to the *real* answer chunks than the bare question's embedding would. You search with text shaped like the target.

The part that surprises everyone: **it does not matter if the hypothetical answer is factually wrong.** HyDE never shows the hypothetical answer to the user and never uses it as content. It uses only its *embedding*, purely as a *search probe* to locate the genuine documents. The real, retrieved chunks — not the hypothetical text — are what get passed to the generator.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

hyde_prompt = ChatPromptTemplate.from_template(
    "Write a short, confident, factual-sounding paragraph that directly "
    "answers the question below. Do not hedge or say you are unsure.\n\n"
    "Question: {question}"
)
hyde_generator = hyde_prompt | llm | StrOutputParser()

# 1. Generate a hypothetical answer
hypothetical = hyde_generator.invoke({"question": "What is the refund window?"})
# 2. Search using the HYPOTHETICAL answer's embedding, not the question's
real_chunks = vector_store.similarity_search(hypothetical, k=4)
# 3. The REAL chunks (not the hypothetical text) go to the generator
```

**COMMON MISTAKE**

Believing HyDE is dangerous because "it generates a possibly-wrong answer." It is not, because that generated answer is *never used as an answer* — only its embedding is used, as a search probe to find *real* documents. The final answer is generated from the genuine retrieved chunks exactly as in normal RAG. HyDE's real costs are different and worth naming: it adds an LLM call (latency) to every query, and it works poorly when the LLM knows so little about the topic that its hypothetical answer is not even *shaped* like the real one.

## 7.8 Corrective RAG (CRAG)

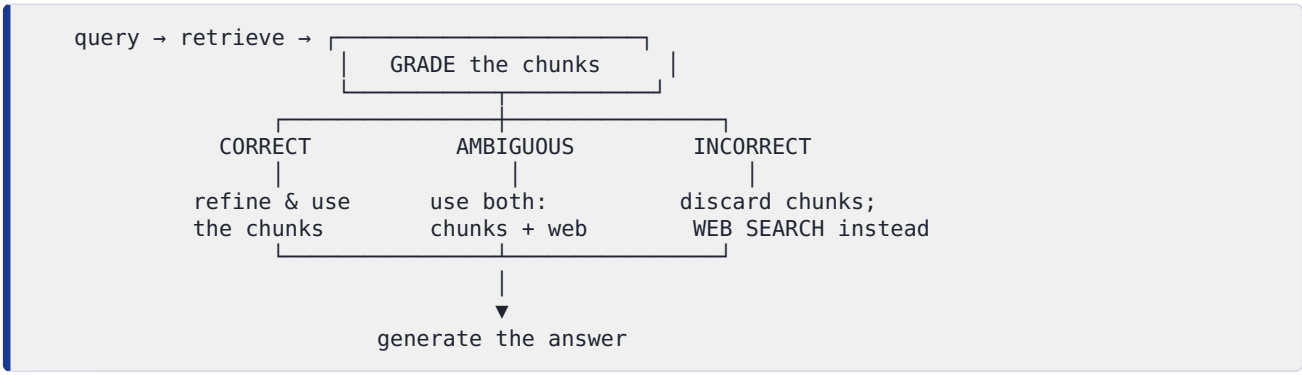
### 7.8.1 The Problem: Blind Trust

Naive RAG has a dangerous assumption baked in: *whatever the retriever returned is good enough to answer from.* But retrievers fail. They return off-topic chunks, outdated chunks, or — when the knowledge base simply lacks the answer — the least-bad irrelevant chunks. Naive RAG passes all of it to the generator anyway, and the generator does its best with garbage, producing a confident wrong answer.

### 7.8.2 The CRAG Solution: Grade, Then Correct

**Corrective RAG (CRAG)** removes the blind trust by inserting an **evaluation step** after retrieval and *before* generation. A grader — typically an LLM acting as a judge — assesses the retrieved chunks against the query and assigns one of three verdicts:

- **Correct** — the retrieved chunks are relevant and sufficient. Proceed to generation, optionally after a *refinement* step that strips each chunk down to its most relevant parts.
- **Incorrect** — the retrieved chunks are irrelevant or insufficient. **Do not generate from them.** Trigger a *fallback*: most characteristically, a **web search** to fetch fresh, relevant information from outside the knowledge base.
- **Ambiguous** — the grader is unsure. Hedge: combine the retrieved chunks *and* a web-search fallback, getting the benefit of both.




CRAG's defining contribution is the **fallback to an external source**: it recognises when its own knowledge base has failed and *goes elsewhere* rather than hallucinating from bad chunks. This is a genuine self-correction loop — and, because it involves a branching decision (which of three paths to take), it is a natural fit for the graph-based control flow of Module 8.

## 7.9 Self-RAG

### 7.9.1 The Problem: Two Unasked Questions

Naive RAG never asks two important questions. First, *should I retrieve at all?* — it retrieves for every query, even "*hello*" or "*what is 2 + 2?*", wasting time and adding noise. Second, *is the answer I just generated actually supported by the evidence?* — it ships the answer without checking it against the chunks it was given.

### 7.9.2 The Self-RAG Solution: Reflect at Every Step

**Self-RAG (Self-Reflective RAG)** makes the system reflect on its own process at several points. The original technique trains the model to emit special **reflection tokens**, but the

*concept* — and a practical approximation using ordinary LLM-as-judge steps — involves four reflective decisions:

1. **Retrieve on demand.** First decide *whether* this query even needs retrieval. Simple, self-contained queries skip it; knowledge-seeking queries trigger it. This makes the system **adaptive** — it does not blindly retrieve every time.
2. **Grade relevance.** If chunks were retrieved, judge each one's relevance and keep only the useful ones (similar to CRAG's grading).
3. **Grade faithfulness (support).** After generating an answer, check whether each claim in it is actually *supported* by the retrieved chunks. An unsupported claim is a hallucination to be removed or revised.
4. **Grade usefulness.** Finally, judge whether the answer actually *addresses the user's question* well.

If a check fails, the system loops back — retrieves again, or regenerates. The contrast with CRAG is worth stating clearly: **CRAG self-corrects the *retrieval; Self-RAG additionally self-corrects the generation*** by verifying the answer against the evidence before delivering it. Self-RAG is the more thorough of the two — and, again, its loops and branches make it a Module 8 (LangGraph) construction.

> **INTERVIEW INSIGHT**
>
> "Compare Corrective RAG and Self-RAG" is a common advanced question. Both add self-evaluation, but: **CRAG** focuses on *retrieval* — it grades the retrieved chunks and, if they are bad, falls back to an external source such as web search. **Self-RAG** is broader — it also decides *whether retrieval is needed at all* (adaptive retrieval) and, crucially, grades the *generated answer* for faithfulness (is every claim supported by evidence?) and usefulness, looping back to fix failures. In one line: CRAG corrects what you *retrieved*; Self-RAG also corrects what you *wrote*.

# 7.10 Graph RAG

## 7.10.1 The Problem: Connecting Facts

Vector RAG retrieves chunks *independently*. Each chunk is found on its own merits, and the chunks are never linked. This breaks down for two kinds of question:

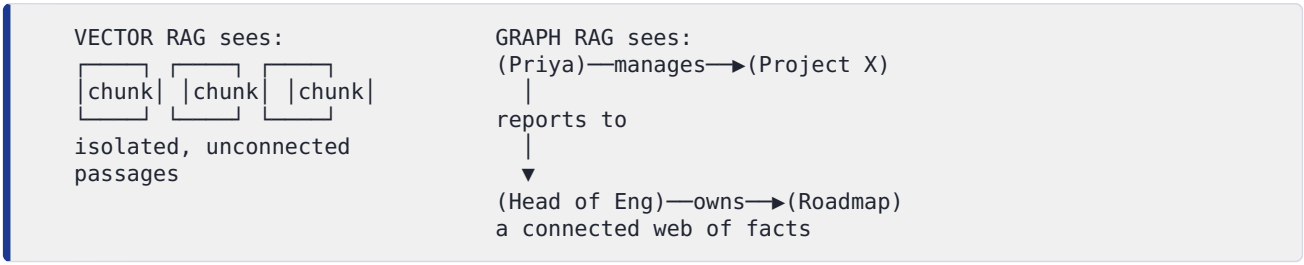
- **Multi-hop questions**, which require chaining facts: "*Which products are managed by people who report to the head of Engineering?*" The answer is assembled from several relationships, possibly spread across many documents. No single chunk contains it.
- **Global / aggregative questions**: "*What are the main themes across all our incident reports?*" This needs a synthesis over the *whole* corpus, not a few similar chunks.

Naive vector RAG, which fetches a handful of locally-similar chunks, simply cannot do this kind of *connective* reasoning.

### 7.10.2 The Graph RAG Solution

**Graph RAG** replaces — or augments — the flat vector store with a **knowledge graph**: a structured network where **entities** (people, products, projects, places) are *nodes*, and **relationships** between them ("*reports to*," "*manages*," "*located in*") are *edges*.

The pipeline changes at both ends. During **indexing**, an LLM reads the documents and *extracts* entities and the relationships between them, assembling the knowledge graph. During **retrieval**, instead of (or alongside) a similarity search, the system *traverses the graph* — following edges from node to node — to gather the connected web of facts a multi-hop question needs. For global questions, a common approach (popularised by Microsoft's GraphRAG) is to detect *communities* of densely connected entities in the graph, summarise each community, and answer from those summaries.




Graph RAG's strength is **relationship-rich and multi-hop reasoning**; its cost is a more complex pipeline — graph construction is an extra, LLM-heavy indexing step. A common production design is **hybrid**: use vector RAG for ordinary fact-lookup questions and Graph RAG for the connective ones.

## 7.11 Multi-modal RAG

### 7.11.1 The Problem: The Answer Is in the Picture

Real documents are not pure text. They contain diagrams, charts, photographs, scanned tables, screenshots. A text-only RAG pipeline is *blind* to all of it — if the answer to "*what was Q3 revenue?*" lives only in a bar chart, text-only RAG cannot find it.

### 7.11.2 The Multi-modal RAG Solution

**Multi-modal RAG** extends the pipeline to retrieve and reason over non-text content. There are two common architectural approaches:

- **Shared multi-modal embedding space.** Use an embedding model that can embed *both* text and images into the *same* vector space. A text query can then directly retrieve relevant images, because their vectors are comparable.

- **Text-description bridging.** Use a multi-modal LLM to *generate a text description* of each image, table, or chart during indexing, then embed those descriptions with an ordinary text embedding model. Retrieval is plain text retrieval over the descriptions; the original image is returned alongside.

Either way, the **generator must be a multi-modal LLM** — one that can accept images as input and reason over them. This is where Gemini fits especially well: Gemini is **natively multi-modal**, able to take images and text together in a single prompt, which makes it a natural generator for multi-modal RAG.

> **WHY THIS MATTERS**
>
> A large fraction of real enterprise knowledge lives in non-text form — financial charts, architecture diagrams, scanned legal exhibits, product photos, slide decks. A text-only RAG system silently ignores all of it and confidently answers from the text alone, missing whatever the figures contained. Multi-modal RAG is increasingly not a luxury but a requirement for any system over real-world business documents.

## 7.12 Agent-Assisted and Adaptive Retrieval

The self-correcting patterns above — CRAG grading and falling back, Self-RAG deciding whether to retrieve and looping to fix answers — all share one quality: the pipeline *makes decisions about its own next step*. Push that idea to its conclusion and you get **agent-assisted retrieval** and **adaptive retrieval systems**.

In an **agentic** RAG system, retrieval is no longer a fixed step — it is a **tool** that an LLM-driven *agent* chooses to use, when and how it sees fit. The agent can decide to retrieve, to retrieve again with a different query, to use a different knowledge source, to do a web search, to ask a clarifying question, or to answer directly without retrieving at all. An **adaptive** retrieval system routes each query down a different path depending on its type — a simple lookup, a multi-hop question, an out-of-scope question — each handled by the most appropriate strategy.

These are the natural endpoint of this module's self-correcting family, and they are exactly the subject of **Module 8 — Agentic RAG with LangGraph**. CRAG and Self-RAG are, in effect, *specific, fixed* agentic workflows; Module 8 gives you the general tools — LangGraph's states, nodes, edges, and conditional routing — to build any of them.

## 7.13 Choosing a Pattern: Enterprise-Grade RAG Architectures

No pattern is universally best. The skill is matching the pattern to the failure. Use this table as a decision guide:

| If naive RAG is failing because... | The pattern to reach for | Family |
|---|---|---|
| Users phrase the same need many ways; recall is patchy | RAG Fusion (multi-query + RRF) | Retrieval-enhancement |
| Short questions miss longer answer passages | HyDE | Retrieval-enhancement |
| The retriever sometimes returns irrelevant chunks and the system answers anyway | Corrective RAG (grade + web-search fallback) | Self-correcting |
| The model hallucinates claims not supported by the evidence | Self-RAG (faithfulness grading) | Self-correcting |
| Retrieval runs even for trivial queries; or answers go unverified | Self-RAG (adaptive + reflective) | Self-correcting |
| Questions need multi-hop reasoning or whole-corpus synthesis | Graph RAG | Retrieval-enhancement |
| Key information lives in images, charts, or scanned tables | Multi-modal RAG | Retrieval-enhancement |
| Retrieval needs to be a flexible, decision-driven step | Agentic RAG (Module 8) | Self-correcting / adaptive |

An **enterprise-grade RAG architecture** rarely uses one pattern; it composes several. A realistic high-end design might: rewrite the query and run **RAG Fusion** for strong retrieval; apply **HyDE** for hard queries; **grade** the results CRAG-style and **fall back to web search** when they are weak; **verify** the generated answer Self-RAG-style for faithfulness before delivery; route relationship-heavy questions to a **Graph RAG** component; handle image-bearing documents with **Multi-modal RAG**; and wrap the whole thing in an **agentic** controller (Module 8) that decides which path each query takes. The patterns are composable layers, not competitors.

**BEST PRACTICE**

Do not start with an advanced pattern. Build naive RAG first, measure it (Module 9), and identify *which specific failure mode* is hurting you most. Then add the *one* pattern that targets that failure, and measure again. Each pattern adds latency, cost, and complexity; adopt it only when a measured failure justifies it. A team that bolts on RAG Fusion, HyDE, CRAG, and Graph RAG simultaneously on day one cannot tell what helped, has a system that is slow and expensive, and cannot debug it. Patterns are earned, one measured improvement at a time.

## 7.14 Workflow

The mental model for this module:

1. **Build and measure naive RAG first.** You cannot choose a pattern without knowing your failure mode.
2. **Diagnose the failure.** Query-answer mismatch? Bad retrieval being trusted? Unverified hallucinations? Multi-hop questions? Non-text content?
3. **Select the matching pattern** from the decision table — and only that one.
4. **Implement it.** Retrieval-enhancement patterns (RAG Fusion, HyDE) fit cleanly into an LCEL chain. Self-correcting patterns (CRAG, Self-RAG) involve branches and loops — implement these with LangGraph (Module 8).
5. **Measure the improvement** (Module 9). Keep the pattern only if its benefit justifies its added latency, cost, and complexity.
6. **Repeat** for the next-biggest failure mode, composing patterns into a layered enterprise architecture.

## 7.15 Step-by-Step Implementation and Code

We will fully implement the two patterns that fit naturally into a linear chain — **RAG Fusion** and **HyDE** — and include the **retrieval grader** that is the core building block of Corrective RAG. The branching, looping versions of CRAG and Self-RAG are built in Module 8 with LangGraph. Save the following as `module7_patterns.py`.

```
"""
module7_patterns.py
Advanced RAG patterns: RAG Fusion (multi-query + RRF), HyDE, and the
retrieval grader that powers Corrective RAG.
Stack: Python + LangChain + Gemini + FAISS.
Install: pip install langchain langchain-community langchain-google-genai \
         faiss-cpu python-dotenv
"""

import os
from dotenv import load_dotenv
from langchain_google_genai import (
    GoogleGenerativeAIEmbeddings, ChatGoogleGenerativeAI)
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()
embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)

docs = [
    Document(page_content="Customers may return any item within 30 days of "
             "delivery for a full refund, provided it is unused."),
    Document(page_content="Refunds are processed to the original payment "
             "method within 5 to 7 business days."),
    Document(page_content="The annual leave entitlement for full-time staff "
             "is 24 days per calendar year."),
]
vector_store = FAISS.from_documents(docs, embeddings)


# ============ PATTERN 1: RAG FUSION (multi-query + RRF) ============

def generate_query_variations(question, n=4):
    """Use the LLM to produce n alternative phrasings of the question."""
    prompt = ChatPromptTemplate.from_template(
        "Generate {n} different search queries that capture the intent of "
        "the question below. Return ONE query per line, nothing else.\n\n"
        "Question: {question}"
    )
    chain = prompt | llm | StrOutputParser()
    text = chain.invoke({"n": n, "question": question})
    return [line.strip() for line in text.splitlines() if line.strip()]


def reciprocal_rank_fusion(ranked_lists, k=60):
    """Merge several ranked lists of Documents using RRF."""
    scores, doc_map = {}, {}
    for ranked_list in ranked_lists:
        for rank, doc in enumerate(ranked_list, start=1):
            key = doc.page_content
            doc_map[key] = doc
            scores[key] = scores.get(key, 0.0) + 1.0 / (k + rank)
    ranked = sorted(scores.items(), key=lambda kv: kv[1], reverse=True)
    return [doc_map[key] for key, _ in ranked]


def rag_fusion_retrieve(question, k=3):
    """RAG Fusion: variations -> retrieve each -> fuse with RRF."""
    variations = generate_query_variations(question)
    ranked_lists = [
        vector_store.similarity_search(q, k=k) for q in variations
    ]
    return reciprocal_rank_fusion(ranked_lists)
```

```python
# ============ PATTERN 2: HyDE (Hypothetical Document Embeddings) ============

hyde_generator = (
    ChatPromptTemplate.from_template(
        "Write a short, confident, factual-sounding paragraph that directly "
        "answers the question. Do not hedge.\n\nQuestion: {question}"
    ) | llm | StrOutputParser()
)

def hyde_retrieve(question, k=3):
    """HyDE: generate a hypothetical answer, search with ITS embedding."""
    hypothetical_answer = hyde_generator.invoke({"question": question})
    return vector_store.similarity_search(hypothetical_answer, k=k)


# ============ BUILDING BLOCK FOR CRAG: the retrieval grader ============

retrieval_grader = (
    ChatPromptTemplate.from_template(
        "You are grading whether a document is relevant to a question. "
        "Answer with exactly one word: 'yes' or 'no'.\n\n"
        "Question: {question}\nDocument: {document}"
    ) | llm | StrOutputParser()
)

def grade_documents(question, retrieved_docs):
    """CRAG core: keep only documents the grader marks relevant."""
    relevant = []
    for doc in retrieved_docs:
        verdict = retrieval_grader.invoke(
            {"question": question, "document": doc.page_content}
        ).strip().lower()
        if "yes" in verdict:
            relevant.append(doc)
    return relevant   # if empty -> CRAG would trigger a web-search fallback


if __name__ == "__main__":
    q = "How long do I have to send something back?"

    print("=== RAG FUSION ===")
    for d in rag_fusion_retrieve(q):
        print(" -", d.page_content)

    print("\n=== HyDE ===")
    for d in hyde_retrieve(q):
        print(" -", d.page_content)

    print("\n=== CRAG retrieval grading ===")
    raw = vector_store.similarity_search(q, k=3)
    good = grade_documents(q, raw)
    print(f"  {len(good)} of {len(raw)} chunks judged relevant.")
    if not good:
        print("  -> CRAG would now fall back to web search.")
```

## 7.16 Line-by-Line Code Explanation

**Setup.** The Gemini embedding and chat models are created once; `temperature=0` keeps every transformation and grading decision deterministic. The three documents include two about refunds and one unrelated chunk about leave, so the grader has something to reject.

**Pattern 1 — RAG Fusion.** `generate_query_variations` prompts the LLM for *n* alternative phrasings of the question and splits the response into a list — this is the multi-query step. `reciprocal_rank_fusion` is the heart of the pattern: it walks every ranked list, and for each document adds `1/(k + rank)` to that document's running score, keyed by content; documents that appear high across multiple lists accumulate the largest totals. It then sorts by score and returns the fused ranking. `rag_fusion_retrieve` ties them together: generate variations, run a similarity search for each, and fuse all the result lists with RRF. Note the deliberate contrast with Module 6's multi-query, which merely pooled — here the *fusion* step re-ranks by consensus.

**Pattern 2 — HyDE.** `hyde_generator` is a small LCEL chain that asks the LLM for a confident, answer-shaped paragraph — explicitly instructed not to hedge. `hyde_retrieve` then does the HyDE move: it generates that hypothetical answer and passes *it* — not the original question — to `similarity_search`. The vector store embeds the hypothetical answer and finds real chunks near it. The user's question text never touches the search; only its hypothetical-answer proxy does. The real chunks returned are what a generator would then use.

**CRAG building block — the retrieval grader.** `retrieval_grader` is an LCEL chain that asks the LLM a strict yes/no question: is this document relevant to the query? `grade_documents` runs the grader over every retrieved chunk and keeps only those marked relevant. This *is* the core of Corrective RAG: the explicit, post-retrieval evaluation step. The crucial line is the comment on the return value — *if the relevant list is empty, CRAG would trigger a web-search fallback.* That branching decision ("relevant chunks exist → generate" versus "none exist → fall back") is exactly the kind of conditional control flow that LangGraph is built for, which is why the full CRAG loop is completed in Module 8.

**The `__main__` block** runs all three on the deliberately awkward, vaguely-phrased query "*How long do I have to send something back?*" — a query that does not contain the words "refund" or "return policy," so it stresses retrieval. You can watch RAG Fusion and HyDE both recover the refund chunks, and watch the grader correctly keep the refund chunks and reject the unrelated leave chunk.

> **COMMON MISTAKE**
>
> Trying to force Corrective RAG and Self-RAG into a single linear LCEL chain. Their defining feature is *branching and looping* — "if the chunks are bad, go do something else"; "if the answer is unsupported, regenerate." A linear chain has no clean way to express "if / else" or "loop back." Attempting it produces tangled, unmaintainable code. The right tool is a *state graph*, and that is precisely what Module 8's LangGraph provides. Recognising *this pattern needs a graph, not a chain* is itself an important architectural skill.

## 7.17 Common Mistakes

- **Reaching for advanced patterns before measuring naive RAG.** You cannot pick the right pattern without knowing your failure mode. Build naive, measure, diagnose, then choose.
- **Adopting many patterns at once.** Each adds latency, cost, and complexity. Add one, measure, keep or discard, repeat.
- **Thinking HyDE is unsafe.** Its hypothetical answer is only a search probe; it is never shown to the user or used as content.
- **Confusing multi-query with RAG Fusion.** Both generate query variations; only RAG Fusion fuses the results with RRF to reward cross-query consensus.
- **Confusing CRAG with Self-RAG.** CRAG corrects *retrieval* (grade chunks, fall back to web search). Self-RAG also corrects *generation* (verify the answer is faithful and useful) and decides *whether to retrieve at all*.
- **Forcing self-correcting patterns into linear chains.** CRAG and Self-RAG branch and loop — they need a state graph (Module 8), not an LCEL chain.
- **Using Graph RAG for everything.** Graph RAG shines on multi-hop and aggregative questions; for ordinary fact lookup it is needless complexity. Hybrid (vector + graph) is usually best.
- **Deploying text-only RAG over image-heavy documents.** The system will silently ignore charts, diagrams, and scanned tables and answer from text alone.

## 7.18 Best Practices

- **Start with naive RAG, measure, then add patterns** to fix specific measured failures — one at a time.
- **Match the pattern to the failure mode** using the decision table; do not adopt patterns speculatively.
- **Use RAG Fusion** when users phrase needs in many ways; understand RRF fuses by rank, not score.
- **Use HyDE** for query-answer asymmetry — but accept its extra LLM call and weak performance on topics the LLM knows nothing about.
- **Use CRAG-style grading** to stop the system answering from bad retrieval, with a sensible fallback such as web search.
- **Use Self-RAG-style faithfulness checks** to catch unsupported claims before they reach the user.
- **Implement self-correcting patterns with LangGraph** (Module 8), not linear chains.

- **Compose patterns into layered enterprise architectures**, and treat every added pattern as a cost to be justified by measurement.

## 7.19 Real-World Applications

- **Customer support and FAQ bots** — RAG Fusion handles the many ways users describe the same problem; CRAG's web-search fallback covers questions the help centre never anticipated.
- **Enterprise knowledge assistants** — Self-RAG's faithfulness checks prevent the bot from inventing company policy; HyDE improves recall on terse internal queries.
- **Legal and financial research** — Graph RAG connects entities across filings and case law for multi-hop questions; faithfulness verification is essential where unsupported claims carry real risk.
- **Scientific and medical RAG** — Graph RAG links findings across papers; CRAG grading guards against answering from irrelevant retrievals.
- **Document intelligence over reports and presentations** — Multi-modal RAG reads the charts, diagrams, and scanned tables that text-only systems ignore.
- **Competitive and market intelligence** — CRAG's external fallback keeps answers current when the internal knowledge base is stale.

## 7.20 Mini Project: A Pattern Comparison Study

Turn this module into measured understanding:

1. Build a naive RAG pipeline (Module 1) over a knowledge base of your choice. This is your baseline.
2. Assemble a test set of about fifteen questions, deliberately including: some phrased very differently from the documents' wording, some that are terse, and some whose answer requires connecting two facts.
3. Implement **RAG Fusion** and **HyDE** from `module7_patterns.py`. For each test question, compare the chunks retrieved by naive RAG, RAG Fusion, and HyDE. Record where each advanced pattern recovers a relevant chunk that naive RAG missed.
4. Implement the **CRAG retrieval grader**. Add a question whose answer is genuinely *absent* from your knowledge base, and confirm the grader rejects all retrieved chunks — the trigger point for a fallback.
5. **Reason about Graph RAG.** Take your two-fact "connecting" questions and explain, in writing, why naive vector RAG struggles with them and how a knowledge graph would help. (Full Graph RAG implementation is an optional stretch goal.)

6. **Write up a recommendation.** For *this* knowledge base and *this* test set, which patterns measurably helped, at what cost in latency, and which would you deploy? This written judgment — pattern selection backed by measurement — is the real skill the module teaches.

## 7.21 Chapter Summary

- **Naive RAG** is a linear, single-pass, blind-trust pipeline. **Advanced RAG patterns** are named re-architectures, each removing a specific failure mode. They form two families: **retrieval-enhancement** patterns and **self-correcting / adaptive** patterns.
- **RAG Fusion** = multi-query retrieval + **Reciprocal Rank Fusion (RRF)**. RRF merges several ranked lists by summing `1/(k + rank)` per document — using *rank*, not raw score — so chunks that rank highly across multiple query variations rise to the top.
- **HyDE** fixes query-answer asymmetry: it has the LLM generate a *hypothetical answer*, embeds *that*, and searches with it — because a fake answer is shaped more like a real answer passage than the question is. The hypothetical text is only a search probe; it is never shown to the user.
- **Corrective RAG (CRAG)** inserts a grading step after retrieval: chunks judged *correct* are used, *incorrect* triggers a fallback (typically web search), *ambiguous* uses both. It self-corrects the **retrieval**.
- **Self-RAG** reflects at several points: whether to retrieve at all (adaptive), whether chunks are relevant, whether the generated answer is *faithful* (supported by evidence), and whether it is *useful* — looping to fix failures. It self-corrects the **generation** as well as retrieval.
- **Graph RAG** replaces or augments the flat vector store with a **knowledge graph** of entities and relationships, enabling **multi-hop** and **aggregative** reasoning that independent-chunk vector RAG cannot do.
- **Multi-modal RAG** extends retrieval and generation to images, charts, and tables — via a shared multi-modal embedding space or by generating text descriptions of visuals — and needs a multi-modal generator such as Gemini.
- Self-correcting and adaptive patterns lead naturally to **agentic RAG** (Module 8): a pipeline that decides its own next step is an agent.
- No pattern is universally best. Build naive RAG, measure, diagnose the failure, add the one matching pattern, measure again. Patterns compose into layered enterprise architectures — but each must be earned by a measured improvement. Self-correcting patterns are implemented with **LangGraph**, not linear chains.

## 7.22 Practice Exercises

1. **Conceptual.** Explain the difference between the retrieval-enhancement family and the self-correcting family of patterns, naming two members of each, in under 120 words.
2. **RRF by hand.** Two ranked lists. List A: [doc1, doc2, doc3]. List B: [doc2, doc4, doc1]. Using `k = 60`, compute the RRF score of every document and give the final fused ranking.
3. **HyDE.** Explain in your own words why searching with a hypothetical answer can beat searching with the question, and explain precisely why a factually wrong hypothetical answer does not corrupt the final result.
4. **CRAG vs Self-RAG.** For each scenario, say whether CRAG, Self-RAG, or both would address it: (a) the retriever returned chunks about the wrong product; (b) the model's answer contains a claim not present in any retrieved chunk; (c) the system retrieves documents even for the query "thanks!".
5. **Graph RAG.** Write two questions over a corpus of company documents that naive vector RAG would handle poorly and Graph RAG would handle well. Explain why for each.
6. **Code.** Run `module7_patterns.py`. Modify `rag_fusion_retrieve` to also print the RRF score of each returned chunk, and report the scores for a query of your choice.
7. **Architecture.** Design, in a short paragraph or diagram, an enterprise RAG architecture for a legal research tool, naming which patterns you would compose and what failure each addresses.

# 7.23 Interview Questions

**BEGINNER**

**Q1. What is "naive RAG" and what are its main weaknesses?**
Naive RAG is the basic linear pipeline: retrieve once, augment, generate. Its weaknesses are query-answer mismatch, blind trust in whatever retrieval returns, unverified generation, no multi-hop reasoning, and being limited to text.

**Q2. What is RAG Fusion?**
A pattern that generates several phrasings of the query, retrieves a ranked list for each, and merges those lists with Reciprocal Rank Fusion — re-ranking so chunks that rank well across multiple query variations rise to the top.

**Q3. What is HyDE?**
Hypothetical Document Embeddings: the LLM generates a hypothetical answer to the question, that answer is embedded, and the search uses its embedding instead of the question's — because a draft answer is shaped more like a real answer passage than the question is.

**Q4. What does Corrective RAG add to naive RAG?**
A grading step after retrieval that judges whether the retrieved chunks are relevant. If they are not, it falls back to an external source such as a web search instead of answering from bad chunks.

**INTERMEDIATE**

**Q5. Explain Reciprocal Rank Fusion and why it uses rank instead of score.**
RRF merges multiple ranked lists by giving each document a score equal to the sum of `1/(k + rank)` over every list it appears in, then sorting by that total. It uses rank rather than raw score because the lists' scores are not comparable — a cosine similarity, a BM25 score, and results from different query variations live on different scales — whereas rank position is comparable across all of them.

**Q6. Why does it not matter if HyDE's hypothetical answer is factually wrong?**
Because the hypothetical answer is used only as a *search probe* — only its embedding is used, to locate *real* documents in the vector store. It is never shown to the user and never used as answer content. The final answer is generated from the genuine retrieved chunks, exactly as in normal RAG.

**Q7. Compare Corrective RAG and Self-RAG.**
Both add self-evaluation. CRAG focuses on retrieval: it grades the retrieved chunks and, if they are inadequate, falls back to an external source. Self-RAG is broader: it also decides whether retrieval is needed at all (adaptive retrieval) and grades the *generated answer* for faithfulness to the evidence and for usefulness, looping back to fix failures. CRAG corrects what you retrieved; Self-RAG also corrects what you wrote.

**Q8. What kinds of question is Graph RAG good at that vector RAG is not?**
Multi-hop questions that require chaining facts across documents (e.g. "which products are run by people reporting to the Engineering head"), and global or aggregative questions that need synthesis over the whole corpus. Vector RAG retrieves chunks independently and cannot connect or aggregate them.

**ADVANCED**

**Q9. Why are Corrective RAG and Self-RAG difficult to implement as a linear chain, and what is the right tool?**

Because their defining feature is branching and looping — "if the retrieved chunks are bad, do a web search instead"; "if the answer is unsupported, regenerate." A linear chain executes a fixed sequence of steps with no clean way to express conditionals or loops. The right tool is a state graph, where nodes are steps and edges (including conditional edges) express branching and looping — which is exactly what LangGraph provides.

**Q10. How does Graph RAG change the indexing and retrieval stages?**

At indexing time, instead of only chunking and embedding, an LLM extracts entities and the relationships between them and assembles a knowledge graph (nodes = entities, edges = relationships); for global questions, communities of densely connected entities may be detected and summarised. At retrieval time, instead of (or alongside) a similarity search, the system traverses the graph — following edges from node to node — to gather the connected web of facts a multi-hop question requires.

**Q11. How would you decide which advanced RAG patterns to adopt for a production system?**

Build naive RAG first and measure it (Module 9). Diagnose the dominant failure mode — query-answer mismatch, bad retrieval being trusted, unverified hallucination, multi-hop questions, non-text content. Add the single pattern that targets that failure, measure the improvement and its added latency and cost, and keep it only if the benefit justifies the cost. Repeat for the next failure. The result is a layered architecture where every pattern is earned by a measured improvement, rather than a speculative pile-up of techniques.

**SCENARIO-BASED**

**Q12. Your RAG support bot answers documented questions well, but when a user asks about something genuinely not in the help centre, it confidently invents an answer. Which pattern fixes this, and how?**

Corrective RAG. Add a grading step that, after retrieval, asks an LLM judge whether the retrieved chunks are actually relevant and sufficient for the question. For an out-of-scope question the retriever returns only weakly related chunks, the grader marks them inadequate, and instead of generating from them the system triggers a fallback — typically a web search for fresh information, or, if no good source exists, an honest "I don't have information on that." This stops the bot answering confidently from irrelevant retrieval. Self-RAG-style faithfulness checking on the generated answer would add a second layer of protection.

**Q13. A financial-analysis RAG system gives weak answers to questions like "how did revenue trends differ across the divisions that the CFO restructured in 2022?" The documents contain the facts, but spread across many filings. What is going wrong and what would you change?**

This is a multi-hop, connective question: it requires identifying which divisions the CFO restructured, then linking each to its revenue trend, then comparing — facts scattered across many documents with no single chunk containing the answer. Naive vector RAG retrieves a few independently-similar chunks and cannot connect them. The fix is Graph RAG: at indexing time extract entities (CFO, divisions, revenue figures, the 2022 restructuring) and their relationships into a knowledge graph; at query time traverse the graph to assemble the connected set of facts. A hybrid design is sensible — keep vector RAG for ordinary fact lookups and route relationship-heavy, multi-hop questions to the Graph RAG component.

**END OF MODULE 7**

You now know the named patterns the field is built on — and, crucially, when to use each. The self-correcting patterns — Corrective RAG and Self-RAG — kept pointing in one direction: pipelines that branch, loop, and decide their own next step. **Module 8 — Agentic RAG with LangGraph** gives you the tool to build exactly those: LangGraph's state, nodes, edges, conditional routing, checkpointing, and memory. There you will turn CRAG and Self-RAG from concepts into running, self-correcting agentic systems.